

Struts2-019/018漏洞分析（CVE-2013-4316）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 漏洞分析

漏洞描述：当Struts版本小于等于2.3.15.1时 `struts.enable.DynamicMethodInvocation` 默认为true，允许动态方法调用，可以通过action前缀调用其它action，例如/user/login.action?action:index!execute，可以访问到namespace为/user，Action为index，Method为execute的方法，从而造成风险

影响版本： Struts 2.0.0 - Struts 2.3.15.1

影响说明： 动态方法调用

环境说明： Struts2（2.3.15.1）、Tomcat（7.0.47）、IDEA（2022.3）

环境搭建： 使用我提供的Struts2-018MV项目，导入自己的IDEA即可

Payload：

```
1 // 访问namespace为/user, Action为index, Method为execute的方法
2 /user/login.action?action:index!execute
3 /user/xxx.action?action:index!execute
```

一点疑惑： 是否可以跨namespace调用action

一、环境介绍

IndexAction

```
1 public class IndexAction extends ActionSupport {
2     public String execute() throws Exception{
3         return "success";
4     }
5 }
```

LoginAction

```
1 public class LoginAction extends ActionSupport {
2     public String execute() throws Exception{
3         return "success";
4     }
5 }
```

struts.xml

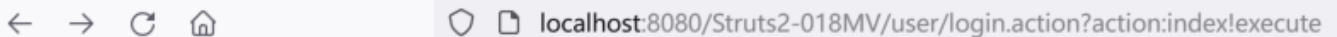
```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE struts PUBLIC
3     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4     "http://struts.apache.org/dtds/struts-2.0.dtd">
5 <struts>
6     <package name="S2-018-Login" extends="struts-default" namespace="/user">
7         <action name="login" class="org.example.LoginAction">
8             <result name="success">/login.jsp</result>
9         </action>
10    </package>
11
12    <package name="S2-018-User" extends="struts-default" namespace="/user">
13        <action name="index" class="org.example.IndexAction">
14            <result name="success">/index.jsp</result>
15        </action>
16    </package>
17 </struts>
```

二、漏洞分析

在分析漏洞之前先看漏洞利用的结果，在浏览器中访问以下URL

```
1 http://localhost:8080/Struts2-018MV/user/login.action?action:index!execute
```

该URL是请求/user/login.action，参数为action:index!execute，正常情况下应该是执行/user/login.action所对应的LoginAction类的execute方法，但若存在漏洞则是执行IndexAction类的execute方法



Welcome Index!

根据访问结果看，确实执行了IndexAction类的execute方法。

根据漏洞描述分析，漏洞的产生是struts处理URL参数时出现的问题，所以需要将struts处理URL参数的核心逻辑梳理清楚，核心逻辑如下

```

1 // struts2入口
2 FilterDispatcher.doFilter()
3 // 获取ActionMapping对象, 内部封装namespace、action、method
4 DefaultActionMapper.getMapping()
5 // 解析分离ActionName与NameSpace
6 DefaultActionMapper.parseNameAndNamespace()
7 // 处理特殊的参数, 例如action: 、 method:
8 DefaultActionMapper.handleSpecialParameters()
9 // 解析分离ActionName, 获取Action名称与method
10 DefaultActionMapper.parseActionName()
11 // 后面的ParametersInterceptor也有处理参数的地方, 但和本漏洞并无关系, 不在分析

```

观察核心逻辑后可以发现主要分析的类是 `DefaultActionMapper`, 分别解释4个方法的逻辑

`DefaultActionMapper.getMapping()` 代码如下

```

1 public ActionMapping getMapping(HttpServletRequest request, ConfigurationManager
  configManager) {
2     ActionMapping mapping = new ActionMapping();
3     // 获取请求URI
4     String uri = getUri(request);
5     //判断URI中是否有;
6     int indexOfSemicolon = uri.indexOf(";");
7     // 如果有;, 则截取;之前的URI
8     uri = (indexOfSemicolon > -1) ? uri.substring(0, indexOfSemicolon) : uri;
9     // 删除action后缀
10    uri = dropExtension(uri, mapping);
11    if (uri == null) {
12        return null;
13    }
14    // 解析分离ActionName与NameSpace
15    parseNameAndNamespace(uri, mapping, configManager);
16    // 处理特殊的参数, 例如action: 、 method:
17    handleSpecialParameters(request, mapping);
18    // 解析分离ActionName, 获取Action名称与method
19    return parseActionName(mapping);
20 }

```

`parseNameAndNamespace()` 方法代码如下

```

1 // 当前处理的是URL中的namespace, 例如/user/login.action?action:index!execute
2 // 此时处理的是/user/login.action, 而非参数action:index!execute
3 protected void parseNameAndNamespace(String uri, ActionMapping mapping,
  ConfigurationManager configManager) {
4     String namespace, name;
5     // /在URI最后出现的位置
6     int lastSlash = uri.lastIndexOf("/");
7     // 如果没有/ , 则namespace为空

```

```

8      if (lastSlash == -1) {
9          namespace = "";
10         name = uri;
11         // 如果/只出现了一次, 且在开头, 则namespace为/
12     } else if (lastSlash == 0) {
13         // ww-1046, assume it is the root namespace, it will fallback to
14         // default
15         // namespace anyway if not found in root namespace.
16         namespace = "/";
17         name = uri.substring(lastSlash + 1);
18         // alwaysSelectFullNamespace在2.3.15.1版本中默认为false
19     } else if (alwaysSelectFullNamespace) {
20         // 最后一个/之前的URI都是namespace, 例如/user/login/login, namespace
    为/user/login
21         namespace = uri.substring(0, lastSlash);
22         name = uri.substring(lastSlash + 1);
23     } else {
24         // 如果没有namespace, 从configManager中尝试查找
25         Configuration config = configManager.getConfiguration();
26         String prefix = uri.substring(0, lastSlash);
27         namespace = "";
28         boolean rootAvailable = false;
29         // 找出配置文件中所有的package
30         for (Object cfg : config.getPackageConfigs().values()) {
31             // 拿到package的namespace
32             String ns = ((PackageConfig) cfg).getNamespace();
33             // 如果ns不为空, 且URI的prefix以ns开头, 且 (prefix的长度与ns相同, 或prefix最后
    一个字符为/
34             if (ns != null && prefix.startsWith(ns) && (prefix.length() ==
    ns.length() || prefix.charAt(ns.length()) == '/')) {
35                 // 如果ns的长度大于namespace的长度 (此时为空)
36                 if (ns.length() > namespace.length()) {
37                     namespace = ns;
38                 }
39             }
40             if ("/".equals(ns)) {
41                 rootAvailable = true;
42             }
43         }
44
45         name = uri.substring(namespace.length() + 1);
46
47         // 如果还是没找到, 就用/作为namespace
48         if (rootAvailable && "/".equals(namespace)) {
49             namespace = "/";
50         }
51     }
52     // 如果允许ActionName中包含/, 则取完整的namespace
53     if (!allowSlashesInActionNames) {

```

```

54         int pos = name.lastIndexOf('/');
55         if (pos > -1 && pos < name.length() - 1) {
56             name = name.substring(pos + 1);
57         }
58     }
59     // 设置namespace
60     mapping.setNamespace(namespace);
61     // 设置name
62     mapping.setName(cleanupActionName(name));
63 }

```

`handleSpecialParameters()` 方法如下

```

1  public void handleSpecialParameters(HttpServletRequest request, ActionMapping
mapping) {
2      // 用于处理特殊的参数
3      Set<String> uniqueParameters = new HashSet<String>();
4      // 获取原始的参数
5      Map parameterMap = request.getParameterMap();
6      // 遍历参数
7      for (Object o : parameterMap.keySet()) {
8          // o为参数名
9          String key = (String) o;
10
11         // 去除图像位置信息
12         if (key.endsWith(".x") || key.endsWith(".y")) {
13             key = key.substring(0, key.length() - 2);
14         }
15
16         // 如果uniqueParameters不存在key
17         // 防止多次处理参数
18         if (!uniqueParameters.contains(key)) {
19             // 从prefixTrie对象中获取前缀为key的ParameterAction
20             ParameterAction parameterAction = (ParameterAction)
prefixTrie.get(key);
21             if (parameterAction != null) {
22                 // 执行ParameterAction的execute方法
23                 // key是URL中?后的参数，这里是处理参数的核心方法
24                 parameterAction.execute(key, mapping);
25                 uniqueParameters.add(key);
26                 break;
27             }
28         }
29     }
30 }

```

`parameterAction.execute()` 方法代码如下

```

1 // ACTION_PREFIX的值为action:
2 put(ACTION_PREFIX, new ParameterAction() {
3     public void execute(String key, ActionMapping mapping) {
4         // 去除前缀, 截取后面的参数值
5         String name = key.substring(ACTION_PREFIX.length());
6         // 如果运行动态方法调用, 则去除! 后面的method
7         if (allowDynamicMethodCalls) {
8             // 找到! 位置
9             int bang = name.indexOf('!');
10            if (bang != -1) {
11                // 取出method
12                String method = name.substring(bang + 1);
13                mapping.setMethod(method);
14                // 取出actionName
15                name = name.substring(0, bang);
16            }
17        }
18        mapping.setName(cleanupActionName(name));
19    }
20 });

```

`parseActionName()` 代码如下

```

1 protected ActionMapping parseActionName(ActionMapping mapping) {
2     if (mapping.getName() == null) {
3         return null;
4     }
5     // 如果运行动态方法调用
6     if (allowDynamicMethodCalls) {
7         // handle "name!method" convention.
8         String name = mapping.getName();
9         // 找到! 出现的位置
10        int exclamation = name.lastIndexOf("!");
11        if (exclamation != -1) {
12            // !前面是actionName
13            mapping.setName(name.substring(0, exclamation));
14            // !后面是method
15            mapping.setMethod(name.substring(exclamation + 1));
16        }
17    }
18    return mapping;
19 }

```

分析完代码后发现 `allowDynamicMethodCalls` 起到了决定性的作用, 当为true时允许动态方法调用

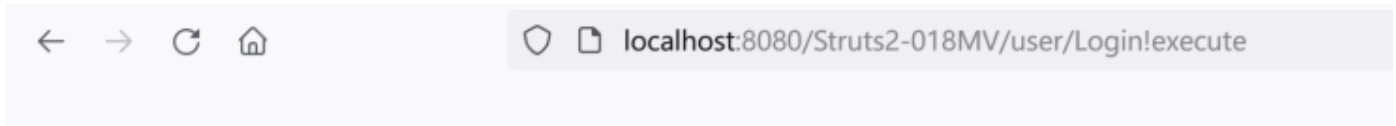
`allowDynamicMethodCalls` 在struts2-core-2.3.15.1.jar中有定义, 默认为true, 此时存在漏洞

```
1  ### Set this to false if you wish to disable implicit dynamic method invocation
2  ### via the URL request. This includes URLs like foo!bar.action, as well as params
3  ### like method:bar (but not action:foo).
4  ### An alternative to implicit dynamic method invocation is to use wildcard
5  ### mappings, such as <action name="*/" method="{2}" class="actions.{1}">
6  struts.enable.DynamicMethodInvocation = true
```

通过 `struts.enable.DynamicMethodInvocation` 的注释可以发现，struts的动态方法调用还有更灵活的配置方法，struts.xml可以改为如下配置

```
1  <package name="S2-018-Login" extends="struts-default" namespace="/user">
2      <action name="*/" method="{2}" class="org.example.{1}Action">
3          <!-- index.jsp与login.jsp文件名要改为首字母大写 -->
4          <result name="success"/>{1}.jsp</result>
5      </action>
6  </package>
```

浏览器访问 `http://localhost:8080/Struts2-018MV/user/Login!execute` 可以调用LoginAction的execute方法，结果如下，不过这个地方和漏洞利用没有关系，仅做扩展知识讲解



You need to log in!

三、是否可以跨namespace调用action?

先说结论：不可以

但可以实现在权限允许的情况下调用任意namespace的action的任意方法

分析之后发现该漏洞不能跨namespace调用action，因为决定namespace是URL，而非参数，例如下面URL连接，namespace一定是 `/user`，这是无法改变的

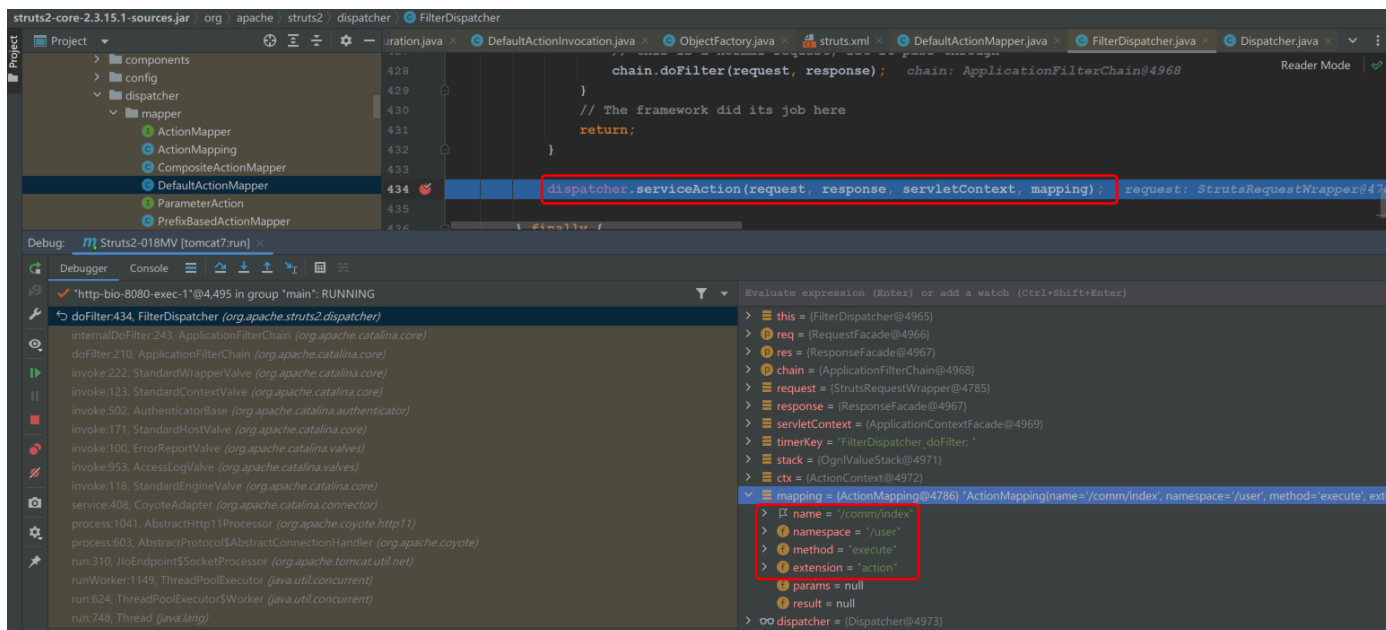
```
1  http://localhost:8080/Struts2-018MV/user/login.action?action:index!execute
```

将struts.xml做如下修改

```
1  <!-- namespace改为/comm -->
2  <package name="S2-018-User" extends="struts-default" namespace="/comm">
3      <action name="index" class="org.example.IndexAction">
4          <result name="success"/>index.jsp</result>
5      </action>
6  </package>
```

如果想通过如下方式访问namespace为/comm，action为index的类是行不通的，因为最终解析的结果是namespace为/comm，action为/comm/index，所以肯定访问不到IndexAction

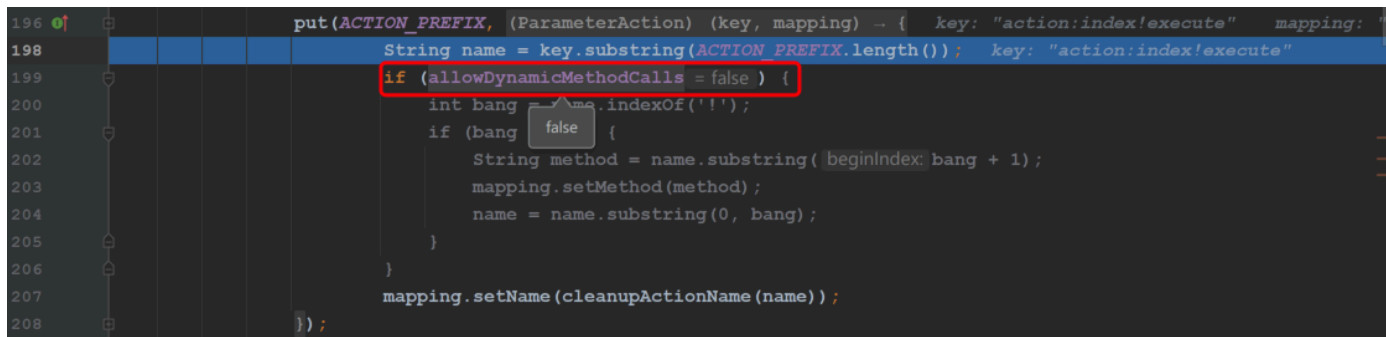
1 `http://localhost:8080/Struts2-018MV/user/login.action?action:/comm/index!execute`



四、修复方法

修复方法可以在sturts.xml中添加如下配置，将动态方法调用关闭

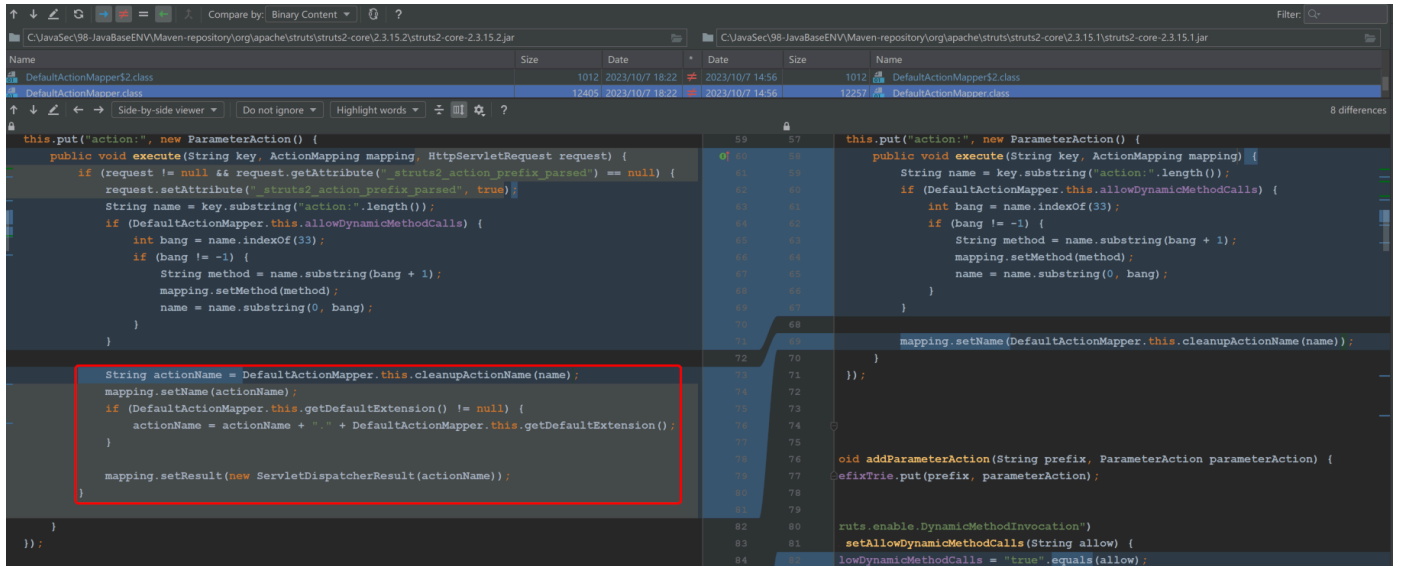
1 `<constant name="struts.enable.DynamicMethodInvocation" value="false"/>`



五、对于补丁的分析

5.1 struts2-core-2.3.15.2补丁分析（Struts2-018漏洞）

官方建议升级到 `struts2-core-2.3.15.2` 版本，虽然该版本可以解决安全问题，但存在bug（可以理解为 Struts2-018漏洞），该版本与 `struts2-core-2.3.15.1` 的核心区别如下图

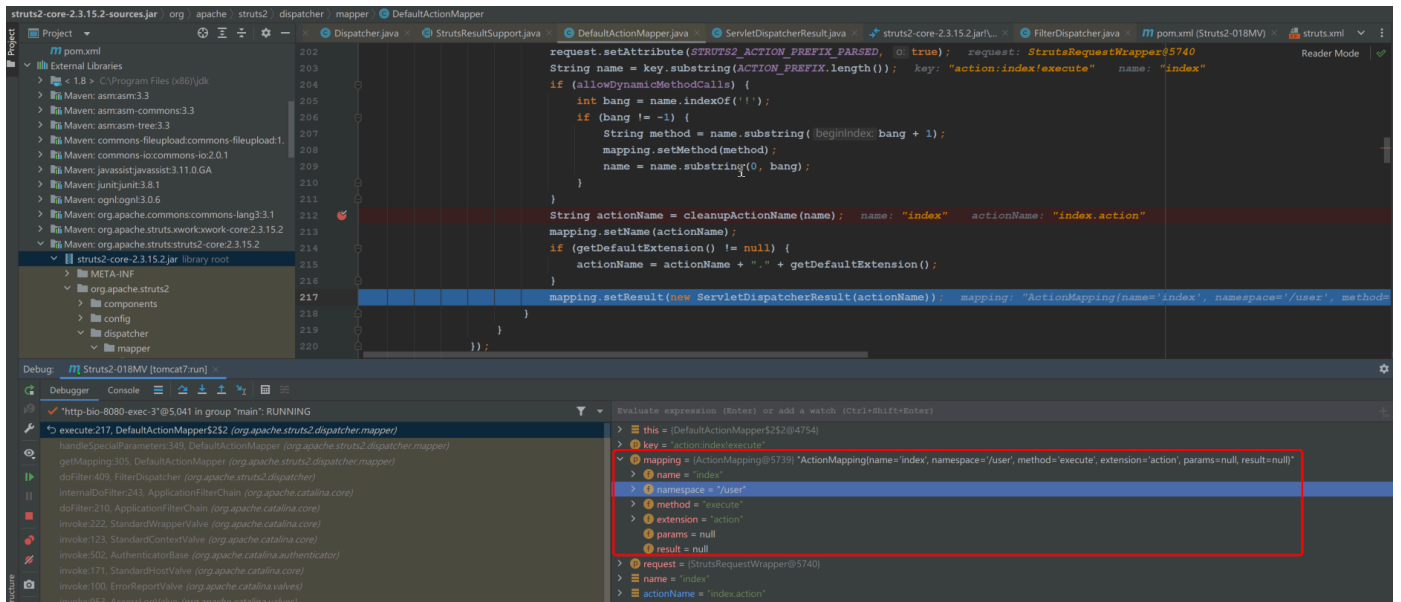


```
1 // 对actionName进行过滤，使用了白名单过滤，正则表达式如下
2 // Pattern.compile("[a-zA-Z0-9._!\\-]*");
3 String actionName = cleanupActionName(name);
4 // ActionMapping配置name
5 mapping.setName(actionName);
6 if (getDefaultExtension() != null) {
7     // 加入默认后缀.action
8     actionName = actionName + "." + getDefaultExtension();
9 }
10 // ActionMapping配置result对象为ServletDispatcherResult
11 // ServletDispatcherResult的父类是StrutsResultSupport，在前面讲Struts2-016漏洞时分析过
12 // 如果location（也就是这里的actionName）可控就可能会执行OGNL表达式
13 // 但actionName被前面的cleanupActionName函数过滤了，所以不存在OGNL表达式的漏洞
14 mapping.setResult(new ServletDispatcherResult(actionName));
```

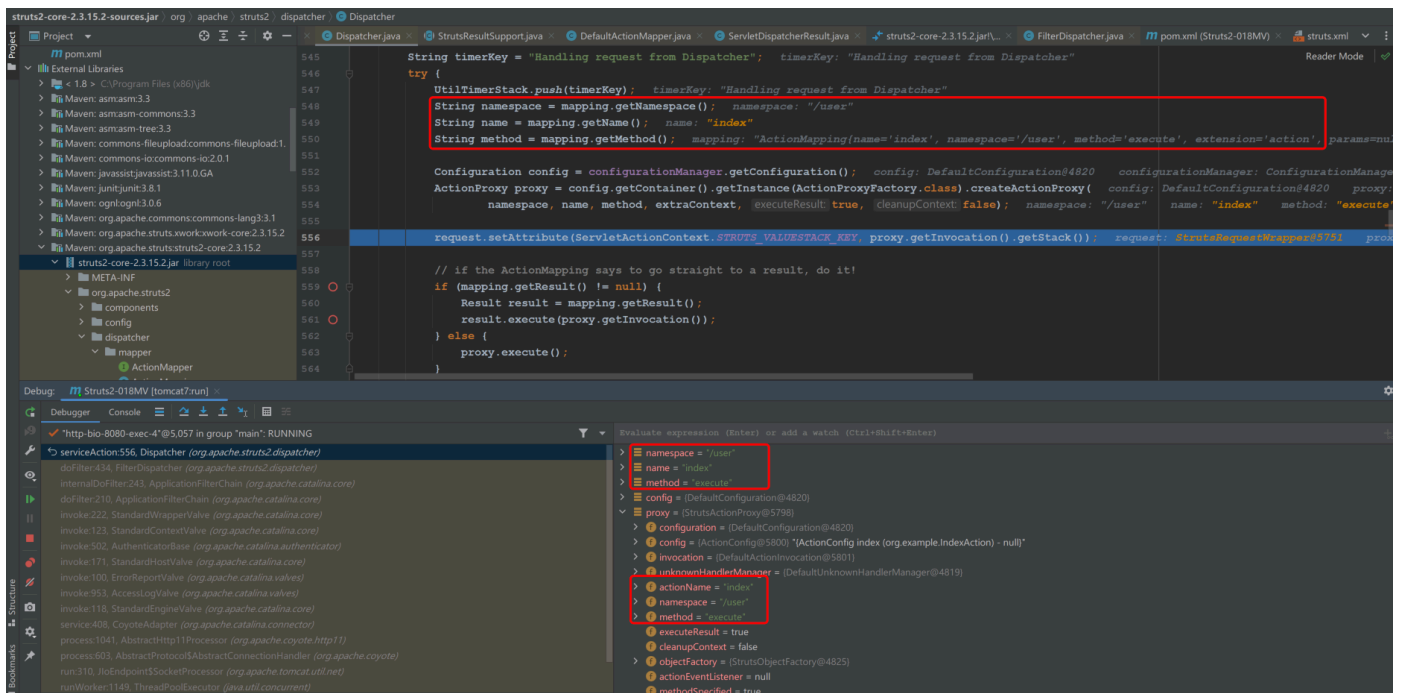
选择升级版本完成修复，测试修复的结果，修复行为如下

- 将Struts2版本升级为 2.3.15.2
- struts.xml中添加 `<constant name="struts.enable.DynamicMethodInvocation" value="true"/>`

请求URL: `localhost:8080/Struts2-018MV/user/login.action?action:index!execute`，预期要执行namespace为/user，action为index，method为execute的方法



在Dispatcher的serviceAction方法执行时可以准确的获取到namespace、name、method，并未发现异常，由于ActionMapping配置result对象为ServletDispatcherResult，所以在Dispatcher的serviceAction方法中会直接执行result.execute而非proxy.execute方法



当代码执行到ServletDispatcherResult.doExecute方法时request与response对象均为空，这就代表无法执行页面的跳转，从而产生异常。

```
124 public void doExecute(String finalLocation, ActionInvocation invocation) throws Exception { finalLocation: "index.action" invocation: DefaultAction
125     if (LOG.isDebugEnabled()) {
126         LOG.debug( msg: "Forwarding to location " + finalLocation);
127     }
128
129     PageContext pageContext = ServletActionContext.getPageContext(); pageContext: null
130
131     if (pageContext != null) {
132         pageContext.include(finalLocation); pageContext: null
133     } else {
134         HttpServletRequest request = ServletActionContext.getRequest(); request: null
135         HttpServletResponse response = ServletActionContext.getResponse(); response: null
136         RequestDispatcher dispatcher = request [NullPointerException].getRequestDispatcher(finalLocation); finalLocation: "index.action" request: null
137
138         //add parameters passed on the location to #parameters
139         // see WW-2120
140         if (StringUtils.isNotEmpty(finalLocation) && finalLocation.indexOf("?") > 0) {
141             String queryString = finalLocation.substring( beginIndex finalLocation.indexOf("?") + 1);
142             Map<String, Object> parameters = getParameters(invocation);
```

request与response对象均为空的原因是ActionContext对HttpServletRequest、HttpServletResponse对象的封装是在StrutsActionProxy.execute方法中完成的，逻辑在Dispatcher.serviceAction方法中

```
1  if (mapping.getResult() != null) {
2      Result result = mapping.getResult();
3      // 执行result对象，此时是没有封装
4      result.execute(proxy.getInvocation());
5  } else {
6      // 在execute方法中完成HttpServletRequest、HttpServletResponse对象的封装
7      proxy.execute();
8  }
```

是不是觉得很奇怪，这里虽然漏洞不能利用了，但感觉确是一个bug，因为无论如何都会在ServletDispatcherResult的136行获取RequestDispatcher时报空指针异常，继续观察下一个版本更新

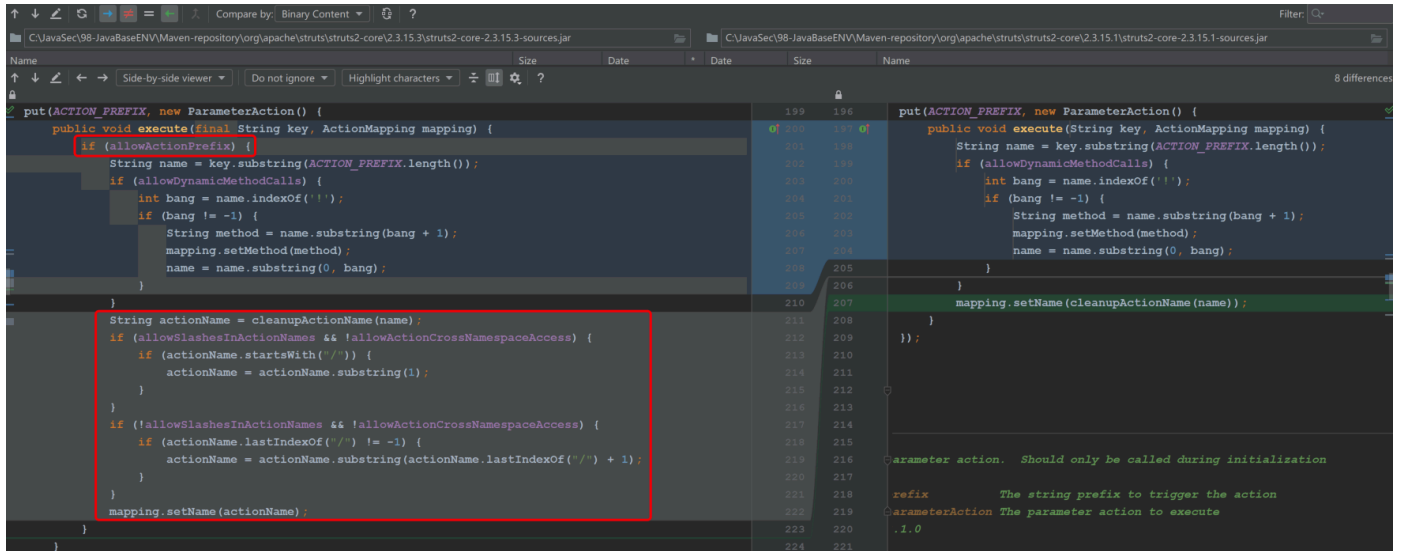
5.2 struts2-core-2.3.15.3补丁分析

struts2-core-2.3.15.3版本中DefaultActionMapper的代码与struts2-core-2.3.15.1版本有非常大的区别

1. allowDynamicMethodCalls默认值变为false，默认不允许动态方法调用
2. 新增allowActionPrefix默认false，默认不允许有action前缀
3. 新增allowActionCrossNamespaceAccess默认false，默认不允许跨namespace访问action

```
protected static final String METHOD_PREFIX = "method:";
protected static final String ACTION_PREFIX = "action:";
protected boolean allowDynamicMethodCalls = false;
protected boolean allowSlashesInActionNames = false;
protected boolean alwaysSelectFullNamespace = false;
protected PrefixTrie prefixTrie = null;
protected Pattern allowedActionNames = Pattern.compile("[a-zA-Z0-9_1/\\\\-]*");
private boolean allowActionPrefix = false;
private boolean allowActionCrossNamespaceAccess = false;
protected List<String> extensions = new ArrayList<String>() {{
    add("action");
    add("");
}};
protected Container container;
```

对于Action前缀的处理同样也发生了变化，不在使用ServletDispatcherResult对象处理result，而是采用之前给ActionMapper设置ActionName的方式（由于allowActionPrefix默认false所以后面的逻辑根本不会执行，而且setAllowActionPrefix方法并没有其它函数调用，所以改变不了）



分析下代码的作用

```
1 // 对actionName进行过滤，使用了白名单过滤，正则表达式如下
2 // Pattern.compile("[a-zA-Z0-9._!\\-]*");
3 String actionName = cleanupActionName(name);
4 // allowSlashesInActionNames默认值为false，决定了actionName中是否有/
5 // allowActionCrossNamespaceAccess默认值为false，决定了action是否可以跨namespace访问
6 // 如果允许actionName中有/，且允许跨namespace访问
7 if (allowSlashesInActionNames && !allowActionCrossNamespaceAccess) {
8     // 如果actionName以/开头，则取全部actionName
9     if (actionName.startsWith("/")) {
10         actionName = actionName.substring(1);
11     }
12 }
13 // 如果不允许actionName中有/，且允许跨namespace访问
14 if (!allowSlashesInActionNames && !allowActionCrossNamespaceAccess) {
15     // 如果如果actionName中有/，则取最后一个/后面的actionName，体现了禁止跨namespace访问的作用
16     if (actionName.lastIndexOf("/") != -1) {
17         actionName = actionName.substring(actionName.lastIndexOf("/") + 1);
18     }
19 }
20 mapping.setName(actionName);
```

分析代码后发现理论上struts2-core-2.3.15.3版本默认是不允许跨namespace访问Action的，而且也不允许使用action前缀来调用其它action，从而修复了漏洞